Lab 11: Strings

CSE 4108

Structured Programming I Lab

Lab Tasks

1. **String Reverse:**

Write your own function in C that mimics the behavior of `strrev` function from `<string.h>`. That means it takes a string as input and reverses the string in place. For instance, if sentence strings holds the string "Hello World" after calling `strrev` on it, sentence will hold "dlroW olleH". The function returns a pointer to the input string to chain functions. The declaration of the function looks like below.

```
char *myStrRev(char *s);
```

Note: You can't use any **built-in functions** for this problem.

2. **Numbered Concatenation:**

Write your own function in C that mimics the behavior of strncat function from <string.h>. That means it takes a string variable, another string literal or variable and maximum number of characters to concatenate as input and concatenates the contents of 2nd string after the contents of the first one. If the first string can accommodate, whole 2nd string is concatenated afterwards and then the function stops. Otherwise, only the stated number of characters are concatenated. Remember the behavior from class and how strncat always puts a '\0' automatically at last. The function returns a pointer to the first string. The declaration looks like below.

```
char *myStrNCat(char *dest, const char *src, int count);
```

Effect of calling this function in main is shown in the following input-output scenario:

Sample Input:

```
Enter Source: Hello
Enter Destination: World
```

Sample Output:

```
Concatenated Destination: HelloWorld
```

Note: You can't use any **built-in functions** for this problem.

3. **Order of Alphabet:**

Write a program that takes 5 strings containing the names of 5 students as input. The program then prints these names in alphabetical order. You can consider that a name will never be more than 10 characters.

Sample Input:

```
Theta
Delta
Alpha
Beta
Gamma
```

Sample Output:

```
Alpha
Beta
Delta
Gamma
Theta
```

Note: You can't use any **built-in functions** for this problem.

4. **IDashing!:**

You're really doing well with Cyber Shepherds. As you're getting acquainted with state-of-the-art techniques to secure the digital world of Utopia, you're closing in on becoming the next Legion. The newest technique in your arsenal is Hashing.

You can consider hashing as a function that would take in a string as input and produce a fixed-size output. That is, no matter what the size of the input length is, your hashing function will always produce a fixed-size output. One important

property of a well-designed hashing function is its a one-way function i.e. you shouldn't get back the original input from the output of the function. This is why passwords are stored after being hashed so that even if the password database is breached, the attacker can't get the original password.

Being thrilled after getting to know about Hashing, you decided to implement your own hash function. You want to show the world that their algorithms are rubbish, your one is far better than theirs. You want to make it unique, so you'd use your ID while designing the hashing algorithm. After passing several sleepless nights trying to figure out that unique algorithm, just when you fell asleep, you got that long, awaited algo in your dream. So, the algorithm is: The ascii value of each character of the input string would be multiplied by last three digits of your ID and then the products would be summed up together. Finally, the length of the input string would be added to the summation. The resulting value will be the output of your hash function. You named your hash function **IDashing**.

To give a concrete example, suppose the input string is "ABC" and your ID is 148. The corresponding ascii values of A, B & C are 65, 66 & 67 respectively and the length of input string is 3. So the calculation would be (65 * 148) + (66 * 148) + (67 * 148) = 29304 + 3 = 29307. Your hash function would output 29307 finally.

Write a C function that implements the IDashing algorithm stated above. The function will take the input string and last 3 digits of your ID as input and will return the final calculated value of the IDashing algorithm. The function declaration looks like below.

```
int iDashing(char *string, int ID);
```

Sample Input:

```
Enter string to hash: HelloFromTheOtherSide
Last 3 digits of ID: 148
```

Sample Output:

```
Hash value: 310229
```